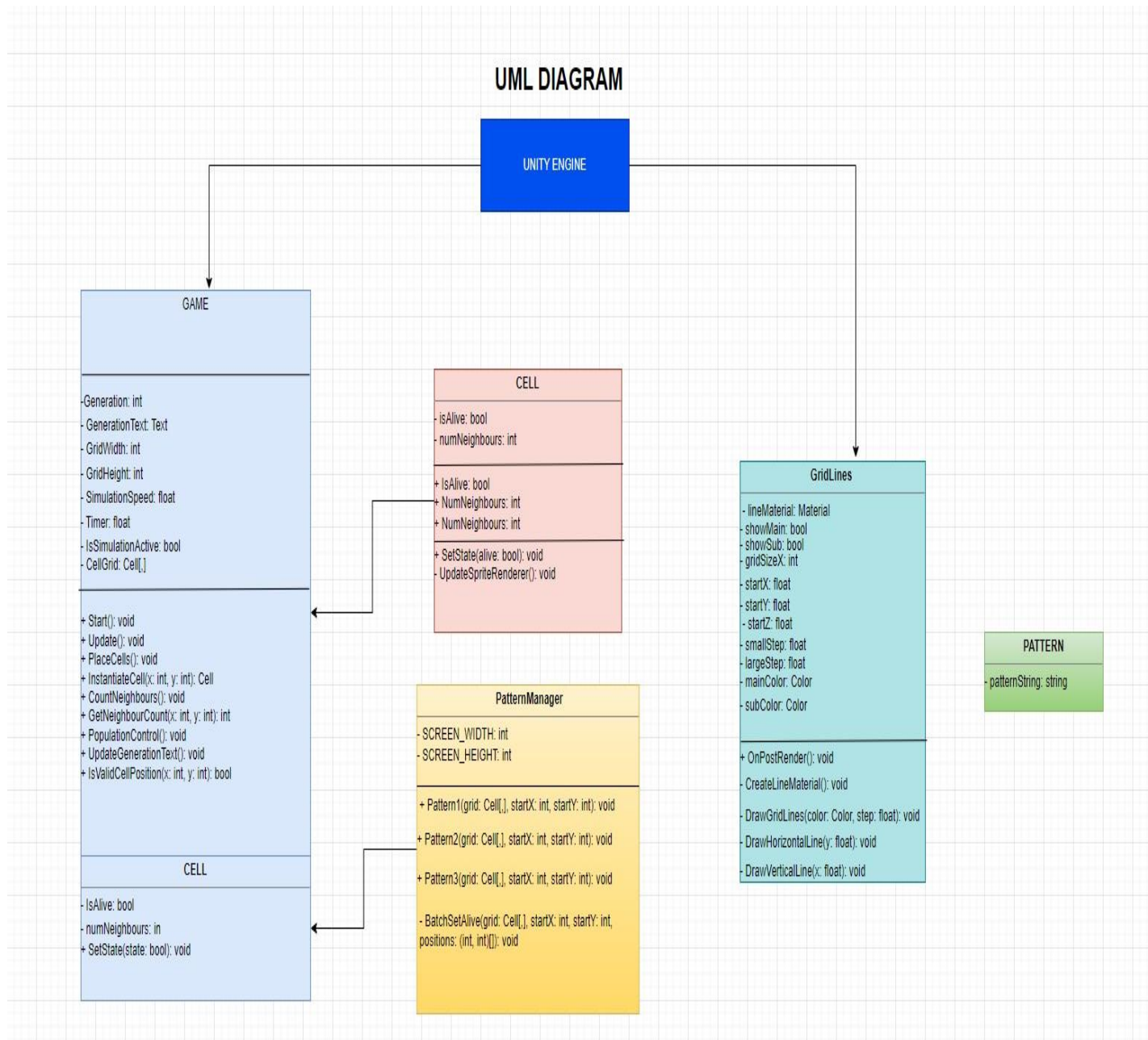


OBJECT-ORIENTED MODELLING AND PROGRAMMING IN ENGINEERING

Submitted by:- SAGGU NAGARAJU

→ FIGURE 1

UML DIAGRAM



NASSI-SCNEIDERMAN DIAGRAMS

→ FIGURE-2

CELL

function CELL() {	🗑️
bool isAlive = false	✚ 🗑️
Int numNeighbours = 0	✚ 🗑️
bool IsAlive => isAlive	✚ 🗑️
int NumNeighbours => numNeighbours	✚ 🗑️
SetState(bool alive)	✚ 🗑️
isAlive = alive	✚ 🗑️
UpdateSpriteRenderer()	✚ 🗑️
UpdateSpriteRenderer()	✚ 🗑️
GetComponent<SpriteRenderer>().enabled = isAlive	✚ 🗑️
}	

→ FIGURE-3

GAME

function GAME() {	🗑️
Int Generation = 0	✚ 🗑️
Text GenerationText	✚ 🗑️
Text GenerationText	✚ 🗑️
Int GridWidth = 40	✚ 🗑️
Int GridHeight = 40	✚ 🗑️
float SimulationSpeed = 0.1f	✚ 🗑️
float Timer = 0	✚ 🗑️
bool IsSimulationActive = false	✚ 🗑️
Cell[,] CellGrid = new Cell[GridWidth, GridHeight]	✚ 🗑️
Start()	✚ 🗑️
PlaceCells()	✚ 🗑️
Update()	✚ 🗑️
If (IsSimulationActive && Timer >= SimulationSpeed)	✚ 🗑️
Timer = 0f	✚ 🗑️
CountNeighbours()	✚ 🗑️
PopulationControl()	✚ 🗑️
Generation++	✚ 🗑️
else	✚ 🗑️
Timer += Time.deltaTime	✚ 🗑️
UpdateGenerationText()	✚ 🗑️

PlaceCells()	+	🔊
for (int y = 0; y < GridHeight; y++)	+	🔊
for (int x = 0; x < GridWidth; x++)	+	🔊
CellGrid[x, y] = InstantiateCell(x, y)	+	🔊
PatternManager.Pattern3(CellGrid, 0, 0)	+	🔊
PatternManager.Pattern1(CellGrid, 0, 0)	+	🔊
PatternManager.Pattern2(CellGrid, 0, 0)	+	🔊
Cell InstantiateCell(int x, int y)	+	🔊
Cell cell = Instantiate	+	🔊
cell.SetState(false)	+	🔊
return cell	+	🔊
CountNeighbours()	+	🔊
for (int y = 0; y < GridHeight; y++)	+	🔊
for (int x = 0; x < GridWidth; x++)	+	🔊
CellGrid[x, y].numNeighbours = GetNeighbourCount(x, y)	+	🔊
int GetNeighbourCount(int x, int y)	+	🔊
int numNeighbours = 0	+	🔊
for (int dx = -1; dx <= 1; dx++)	+	🔊
for (int dy = -1; dy <= 1; dy++)	+	🔊
if (dx == 0 && dy == 0) continue	+	🔊
int nx = x + dx	+	🔊
int ny = y + dy	+	🔊
if (IsValidCellPosition(nx, ny) && CellGrid[nx, ny].IsAlive)	+	🔊
numNeighbours++	+	🔊
return numNeighbours	+	🔊

int Generation = 0	+	🔊
Text GenerationText	+	🔊
Text GenerationText	+	🔊
int GridWidth = 40	+	🔊
int GridHeight = 40	+	🔊
float SimulationSpeed = 0.1f	+	🔊
float Timer = 0	+	🔊
bool IsSimulationActive = false	+	🔊
Cell[,] CellGrid = new Cell[GridWidth, GridHeight]	+	🔊
Start()	+	🔊
PlaceCells()	+	🔊
Update()	+	🔊
if (IsSimulationActive && Timer >= SimulationSpeed)	+	🔊
Timer = 0f	+	🔊
CountNeighbours()	+	🔊
PopulationControl()	+	🔊
Generation++	+	🔊
else	+	🔊
Timer += Time.deltaTime	+	🔊
UpdateGenerationText()	+	🔊

Int GetNeighbourCount(Int x, Int y)	+	🗑
int numNeighbours = 0	+	🗑
for (Int dx = -1; dx <= 1; dx++)	+	🗑
for (Int dy = -1; dy <= 1; dy++)	+	🗑
if (dx == 0 && dy == 0) continue	+	🗑
Int nx = x + dx	+	🗑
Int ny = y + dy	+	🗑
if (IsValidCellPosition(nx, ny) && CellGrid[nx, ny].IsAlive)	+	🗑
numNeighbours++	+	🗑
return numNeighbours	+	🗑
PopulationControl()	+	🗑
for (Int y = 0; y < GridHeight; y++)	+	🗑
for (Int x = 0; x < GridWidth; x++)	+	🗑
bool isAlive = CellGrid[x, y].IsAlive	+	🗑
Anweisung	+	🗑
CellGrid[x, y].SetState....	+	🗑
UpdateGenerationText()	+	🗑
Debug.Log(" Generation: " + Generation)	+	🗑
if (GenerationText != null)	+	🗑
GenerationText.text = "Generation: " + Generation	+	🗑
bool IsValidCellPosition(Int x, Int y)	+	🗑
return x >= 0 && x < GridWidth && y >= 0 && y < GridHeight	+	🗑
}		

→ **FIGURE-4**

GRIDLINES

function GridLines() {	
Material lineMaterial	✚ 🗑
bool showMain = true	✚ 🗑
bool showSub = false	✚ 🗑
int gridSizeX	✚ 🗑
Anweisung	✚ 🗑
float startX	✚ 🗑
float startY	✚ 🗑
float startZ	✚ 🗑
float smallStep	✚ 🗑
float largeStep	✚ 🗑
Color mainColor	✚ 🗑
Color SubColor	✚ 🗑

CreateLineMaterial	✚ 🗑
if (!lineMaterial)	✚ 🗑
lineMaterial = CreateMaterialWithShader()	✚ 🗑
ConfigureMaterialForAlphaBlending(lineMaterial)	✚ 🗑
ConfigureMaterialForDepthWriting(lineMaterial)	✚ 🗑
ConfigureMaterialForBackfaceCulling(lineMaterial)	✚ 🗑
Material CreateMaterialWithShader()	✚ 🗑
var shader	✚ 🗑
var material	✚ 🗑
return material	✚ 🗑
ConfigureMaterialForAlphaBlending(Material material)	✚ 🗑
material.SetInt(SrcBlend)	✚ 🗑
material.SetInt(DstBlend)	✚ 🗑

ConfigureMaterialForDepthWriting(Material material)	✚ 🗑
material.SetInt	✚ 🗑
ConfigureMaterialForBackfaceCulling(Material material)	✚ 🗑
material.SetInt("_Cull", (int)UnityEngine.Rendering.CullMode.Off)	✚ 🗑
OnDisable()	✚ 🗑
DestroyImmediate(lineMaterial)	✚ 🗑
OnPostRender()	✚ 🗑
CreateLineMaterial()	✚ 🗑
lineMaterial.SetPass(0)	✚ 🗑
GL.Begin(GL.LINES)	✚ 🗑
if (showSub) DrawGridLines(subColor, smallStep)	✚ 🗑
if (showMain) DrawGridLines(mainColor, largeStep)	✚ 🗑
GL.End()	✚ 🗑

ConfigureMaterialForBackfaceCulling(Material material)	+	🗑
material.SetInt("_Cull", (int)UnityEngine.Rendering.CullMode.Off)	+	🗑
OnDisable()	+	🗑
DestroyImmediate(lineMaterial)	+	🗑
OnPostRender()	+	🗑
CreateLineMaterial()	+	🗑
lineMaterial.SetPass(0)	+	🗑
GL.Begin(GL.LINES)	+	🗑
if (showSub) DrawGridLines(subColor, smallStep)	+	🗑
if (showMain) DrawGridLines(mainColor, largeStep)	+	🗑
GL.End()	+	🗑
DrawGridLines(Color color, float step)	+	🗑
GL.Color(color)	+	🗑
for (float y = 0; y <= gridSizeY; y += step)	+	🗑
DrawHorizontalLine(y)	+	🗑
DrawVerticalLine(y)	+	🗑
DrawHorizontalLine(float y)	+	🗑
GL.Vertex3(startX, startY + y, startZ)	+	🗑
GL.Vertex3(startX + gridSizeX, startY + y, startZ)	+	🗑
DrawVerticalLine(float x)	+	🗑
GL.Vertex3(startX + x, startY, startZ)	+	🗑
GL.Vertex3(startX + x, startY + gridSizeY, startZ)	+	🗑

→ **FIGURE-5**

PATTERN

function Pattern() {	🗑
class Pattern	↕ 🗑
public string patternString	↕ 🗑
}	

→ **FIGURE-6**

PATTERN MANAGER

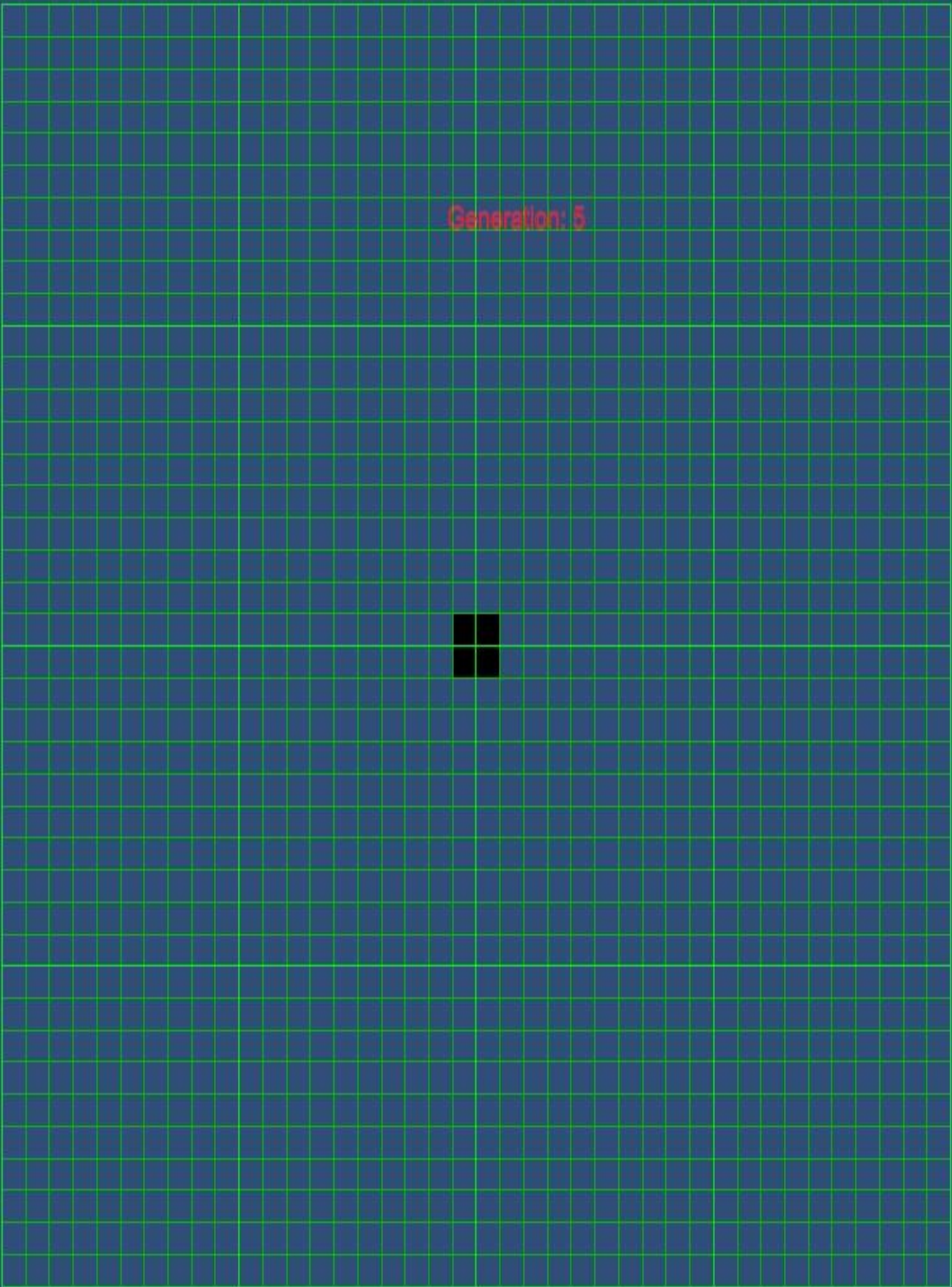
```
function  Pattern Manager( ) {  
  int SCREEN_WIDTH = 40  
  int SCREEN_HEIGHT = 40  
  Pattern1(Cell[,] grid, int startX, int startY)  
    BatchSetActive - Pattern 1 Coordinates  
  Pattern2(Cell[,] grid, int startX, int startY)  
    BatchSetActive - Pattern 2 Coordinates  
  Pattern3(Cell[,] grid, int startX, int startY)  
    BatchSetActive - Pattern 3 Coordinates  
  BatchSetActive(Cell[,] grid, int startX, int startY, params (int, int)[] positions)  
    foreach (var (x, y) in positions)  
      grid[startX + x, startY + y].SetState(true)  
}
```

RESULTS

PATTERN-1

Generation: 1





Generation: 10



Generation: 20



Generation: 50



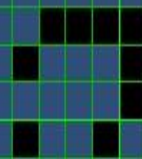
Generation: 5



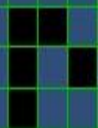
Generation: 10



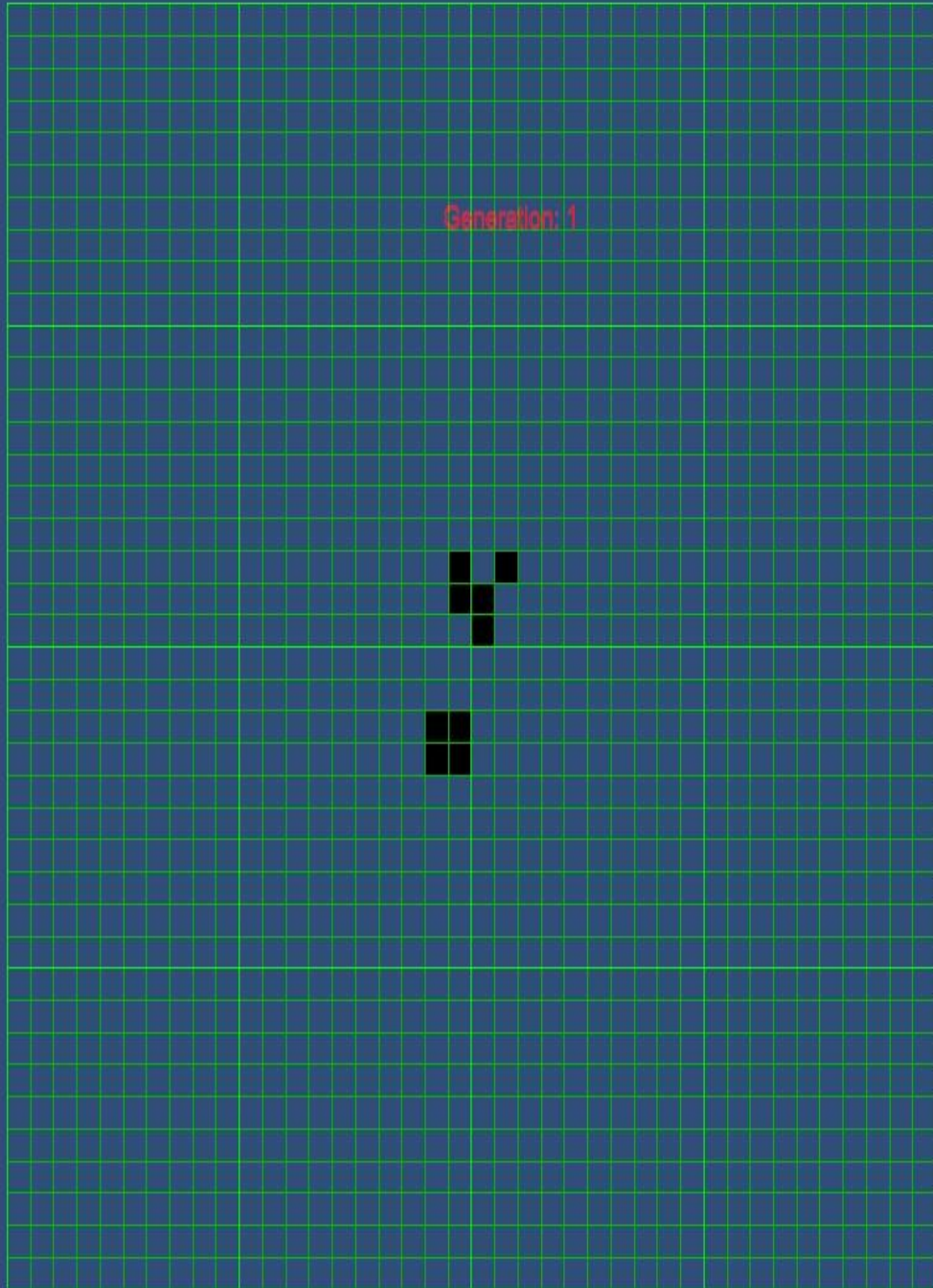
Generation: 20



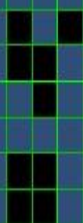
Generation: 50



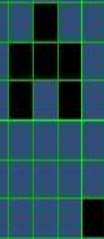
PATTERN-3



Generation: 5



Generation: 10



Generation: 20

